RAPPORT DE PROJET

EduHub

Plateforme d'apprentissage en ligne

Technologie	Laravel (PHP) + Blade + Tailwind CSS
Base de données	MySQL
Déploiement	Docker
Dépôt	https://github.com/jerymy-k/EduHub
Année	2025 - 2026
État	En cours de développement

1. Introduction et Contexte du Projet

1.1 Présentation générale

EduHub est une plateforme d'apprentissage en ligne (LMS - Learning Management System) développée avec le framework Laravel. Ce projet a pour ambition de créer un écosystème éducatif numérique complet permettant aux enseignants de créer et partager des cours, et aux étudiants de les suivre de manière interactive et structurée.

La plateforme répond à un besoin croissant dans le domaine de l'enseignement à distance, notamment après la montée en puissance des outils numériques dans l'éducation. EduHub se positionne comme une solution open-source, flexible et déployable facilement grâce à Docker.

1.2 Objectifs du projet

- Fournir une plateforme intuitive pour la gestion de cours en ligne
- Permettre l'inscription et le suivi de progression des apprenants
- Offrir un tableau de bord administrateur et enseignant complet
- Gérer l'authentification, les rôles et les permissions des utilisateurs
- Permettre le téléchargement et la visualisation de ressources pédagogiques
- Assurer une déployabilité simple via Docker et Render.com

1.3 Périmètre fonctionnel

Le projet couvre trois types d'utilisateurs principaux avec des fonctionnalités distinctes : les administrateurs pour la gestion globale, les enseignants (instructeurs) pour la création de contenu, et les étudiants pour la consommation des cours. Chaque rôle dispose d'un espace dédié avec un tableau de bord personnalisé.

2. Architecture Technique

2.1 Stack technologique

EduHub repose sur un ensemble de technologies modernes et éprouvées dans le développement web :

Composant	Technologie	Rôle
Backend	PHP 8.x + Laravel 11	Framework principal, logique métier, ORM
Frontend	Blade + Tailwind CSS + Vite	Templates serveur, styles, bundler
Base de données	MySQL	Stockage des données persistantes
Auth	Laravel Auth (Breeze)	Authentification et sessions
Déploiement	Docker + infinityfree	Conteneurisation et hébergement cloud
Tests	PHPUnit	Tests unitaires et fonctionnels

2.2 Structure du répertoire Laravel

Le projet suit la structure conventionnelle d'une application Laravel, organisée de la manière suivante :

Dossier / Fichier	Description
app/	Contient les Models, Controllers, Middleware, Policies et Services
bootstrap/	Initialisation du framework Laravel
config/	Fichiers de configuration (BD, mail, auth, cache...)
database/	Migrations, seeders et factories pour la base de données
public/	Point d'entrée web, assets compilés (CSS, JS, images)
resources/	Vues Blade, CSS/JS source, fichiers de traduction
routes/	Définition des routes web et API
storage/	Fichiers uploadés, logs, cache de vues
tests/	Tests PHPUnit (Unit + Feature)
Dockerfile	Configuration de l'image Docker
compose.yaml	Orchestration Docker Compose (app + BD + nginx)
render.yaml	Configuration de déploiement Render.com

2.3 Architecture MVC

Laravel implémente le patron de conception MVC (Model - View - Controller) qui structure clairement les responsabilités :

- Model : Les classes Eloquent représentent les entités métier (User, Course, Lesson, Enrollment...) et gèrent les interactions avec la base de données.
- View : Les templates Blade génèrent le HTML rendu côté serveur, stylisé avec Tailwind CSS.
- Controller : Les controleurs traitent les requêtes HTTP, appellent les services et retournent les vues ou réponses JSON.

3. Modèle de Données

3.1 Entités principales

La base de données d'EduHub est structurée autour des entités suivantes, mises en place via les migrations Laravel :

Entité (Table)	Champs principaux	Description
users	id, name, email, password, role	Utilisateurs de la plateforme (admin, instructor, student)
courses	id, title, description, user_id, category_id, price, status	Cours créés par les instructeurs
lessons	id, course_id, title, content, video_url, order	Leçons constituant un cours
enrollments	id, user_id, course_id, progress, enrolled_at	Inscriptions des étudiants aux cours
categories	id, name, slug, description	Catégories de cours (Informatique, Math...)
reviews	id, user_id, course_id, rating, comment	Avis et notes donnés aux cours
payments	id, user_id, course_id, amount, status	Transactions de paiement (si cours payant)

3.2 Relations entre entités

- Un User (instructor) peut avoir plusieurs Courses (One-to-Many)
- Un Course contient plusieurs Lessons (One-to-Many)
- Un User (student) peut être inscrit à plusieurs Courses via Enrollment (Many-to-Many)
- Un Course appartient à une Category (Many-to-One)
- Un User peut laisser plusieurs Reviews sur plusieurs Courses (Many-to-Many)

4. Fonctionnalités de la Plateforme

4.1 Gestion des utilisateurs et authentification

- Inscription et connexion sécurisée (email + mot de passe hashé)
- Système de rôles : Administrateur, Instructeur, Étudiant
- Gestion du profil : photo, biographie, informations personnelles
- Réinitialisation du mot de passe par email
- Middleware de protection des routes selon les rôles

4.2 Espace Instructeur

- Création et gestion de cours (titre, description, catégorie, niveau, prix)
- Ajout de leçons avec contenu texte, vidéos YouTube/Vimeo et ressources PDF
- Gestion de l'ordre des leçons (drag & drop)
- Tableau de bord : statistiques d'inscription, revenus, notes moyennes
- Publication / dépublication d'un cours

4.3 Espace Étudiant

- Recherche et parcours du catalogue de cours par catégorie
- Inscription aux cours (gratuits ou payants)
- Suivi de la progression leçon par leçon
- Système d'évaluation et de commentaire (notation 1 à 5 étoiles)
- Tableau de bord personnel : cours suivis, progression, certificats

4.4 Panneau d'administration

- Gestion complète des utilisateurs (création, modification, suppression, rôle)
- Validation et modération des cours soumis par les instructeurs
- Gestion des catégories de cours
- Statistiques globales : nombre d'utilisateurs, cours, inscriptions, revenus
- Gestion des signalements et commentaires inappropriés

5. Routes et Contrôleurs

5.1 Organisation des routes

Les routes de l'application sont définies dans le dossier routes/ et organisées selon les fonctionnalités :

Méthode	Route	Contrôleur	Description
GET	/	HomeController	Page d'accueil publique
GET	/courses	CourseController	Liste de tous les cours
GET	/courses/{slug}	CourseController	Détail d'un cours
POST	/courses/{id}/enroll	EnrollmentController	Inscription à un cours
GET	/dashboard	DashboardController	Tableau de bord étudiant
GET	/instructor/courses	InstructorController	Gestion des cours instructeur
POST	/instructor/courses	InstructorController	Création d'un cours
GET	/admin/users	AdminController	Gestion des utilisateurs
GET	/admin/courses	AdminController	Modération des cours

6. Déploiement et Infrastructure

6.1 Containerisation avec Docker

Le projet intègre un Dockerfile et un fichier compose.yaml permettant de lancer l'environnement complet en quelques commandes. L'architecture Docker comprend :

- Un conteneur pour l'application Laravel (PHP-FPM + Nginx)
- Un conteneur pour la base de données MySQL ou PostgreSQL
- Un conteneur pour le serveur web Nginx (reverse proxy)

Commandes de démarrage locales :

```
docker compose up -d
php artisan migrate --seed
```

6.2 Déploiement sur Render.com

Le fichier render.yaml configure le déploiement automatique sur la plateforme cloud Render. Cette approche permet :

- Un déploiement continu depuis la branche main du dépôt GitHub
- La gestion des variables d'environnement (clé d'application, BD, mail)
- La mise à l'échelle automatique selon la charge

6.3 Variables d'environnement critiques

Variable	Description
APP_KEY	Clé de chiffrement de l'application Laravel
DB_CONNECTION	Type de base de données (mysql / pgsql)
DB_HOST, DB_DATABASE, DB_USERNAME, DB_PASSWORD	Connexion à la base de données
MAIL_*	Configuration du serveur SMTP pour les emails
APP_URL	URL de production de l'application

7. Guide d'Installation et de Développement

7.1 Prérequis

- PHP >= 8.1 et Composer
- Node.js >= 18 et NPM
- MySQL 8.0 ou PostgreSQL 14+
- Docker et Docker Compose (optionnel mais recommandé)

7.2 Étapes d'installation

1. Cloner le dépôt : git clone https://github.com/jerymy-k/EduHub.git
2. Installer les dépendances PHP : composer install
3. Installer les dépendances Node : npm install
4. Copier et configurer le fichier d'environnement : cp .env.example .env
5. Générer la clé d'application : php artisan key:generate
6. Exécuter les migrations et les seeders : php artisan migrate --seed
7. Compiler les assets frontend : npm run dev
8. Démarrer le serveur de développement : php artisan serve

7.3 Lancement via Docker

9. Copier le fichier .env.example en .env et configurer les variables
10. Lancer l'ensemble des services : docker compose up -d
11. Exécuter les migrations dans le conteneur : docker compose exec app php artisan migrate --seed
12. Accéder à l'application sur http://localhost:8000

8. Tests et Qualité du Code

8.1 Stratégie de tests

Le projet utilise PHPUnit (intégré dans Laravel) pour les tests automatisés, organisés en deux catégories :

- Tests unitaires (Unit) : Validation des modèles, des services et des fonctions utilitaires de manière isolée
- Tests fonctionnels (Feature) : Simulation de requêtes HTTP complètes (inscription, connexion, création de cours, inscriptions...)

8.2 Exécution des tests

```
php artisan test
```

8.3 Bonnes pratiques intégrées

- Utilisation de factories pour générer des données de test réalistes
- Base de données de test isolée (SQLite in-memory ou base dédiée)
- Validation stricte des formulaires avec les Form Requests Laravel
- Gestion des exceptions et messages d'erreur conviviaux

9. Planning et Organisation du Projet

9.1 Découpage en phases

Phase	Activités	Durée estimée	État
Phase 1	Analyse des besoins, conception BDD, maquettes UI	2 semaines	Terminé
Phase 2	Mise en place du projet Laravel, authentification, gestion des rôles	2 semaines	Terminé
Phase 3	Module cours : CRUD instructeur, catégories, leçons	3 semaines	En cours
Phase 4	Module étudiant : inscription, progression, tableau de bord	2 semaines	À faire
Phase 5	Panneau d'administration, modération, statistiques	2 semaines	À faire
Phase 6	Tests, corrections de bugs, optimisation des performances	1 semaine	À faire
Phase 7	Déploiement Docker + Render.com, documentation finale	1 semaine	À faire

9.2 Gestion du dépôt Git

Le projet est versionné sur GitHub avec une branche principale main. Les TODO files présents dans le dépôt (TODO.md, TODO-50-PARTS.md, TODO-FIX-DASHBOARD.md) servent de backlog de tâches pour guider le développement. Il est recommandé d'adopter le workflow Git Flow pour la suite du projet :

- Branche main : code stable prêt à la production
- Branche develop : intégration continue des nouvelles fonctionnalités
- Branches feature/xxx : développement de chaque fonctionnalité en isolation

10. Sécurité et Bonnes Pratiques

10.1 Mesures de sécurité implémentées

- Protection CSRF sur tous les formulaires (jeton Laravel automatique)
- Validation côté serveur de toutes les entrées utilisateur (Form Requests)
- Hachage des mots de passe avec bcrypt (via Laravel Hash facade)
- Protection contre les injections SQL grâce à l'ORM Eloquent (requêtes préparées)
- Contrôle d'accès basé sur les rôles (Middleware + Policies Laravel)
- Protection XSS via l'échappement automatique dans les templates Blade ({{ }})
- Variables sensibles stockées dans .env (jamais dans le code source)

10.2 Recommandations supplémentaires

- Configurer HTTPS en production (certificat SSL via Let's Encrypt)
- Activer le logging des actions administrateur
- Mettre en place une politique de mots de passe forts
- Exécuter régulièrement composer audit et npm audit

11. Conclusion et Perspectives

11.1 Bilan du projet

EduHub est un projet ambitieux qui couvre l'ensemble du cycle de développement d'une application web moderne : conception de la base de données, développement backend avec Laravel, interface utilisateur avec Blade et Tailwind CSS, et déploiement via Docker. Le choix de Laravel garantit une architecture robuste, maintenable et sécurisée.

11.2 Perspectives d'évolution

- Intégration d'un système de paiement en ligne (Stripe, PayPal)
- Ajout d'un module de quiz et d'évaluation avec certificats automatiques
- API REST complète pour une future application mobile (Laravel Sanctum)
- Système de messagerie interne entre étudiants et instructeurs
- Fonctionnalité de cours en direct (intégration Zoom ou Jitsi)
- Optimisation SEO et mode hors-ligne (PWA)
- Intégration de l'IA pour des recommandations de cours personnalisées

EduHub - Apprendre sans limites, enseigner sans frontières.